Plaintext	:	e4ff78
Hash	:	c1b2d3114d814f41638bdafcaeb8e057
Reduced result	:	c1b2d3

Plaintext	:	c1b2d3
Hash	:	6e929c2625cc49bc50d2d1ae845f9cf7

This chain will be stored as a combination of "a6tyv2" and "6e929c2625cc49bc50d2d1ae845f9cf7"

So how will this help with cracking hashes?

Here is how it will work. When you are looking for a plaintext for a particular hash, the cracking algorithm will first look to see if the hash exists in the table. If it does, we know that the chain for the matching hash contains the plaintext. If we don't find the hash, we reduce the hash to a plaintext and hash that again. We then look for this hash in the table. We continue this process of reducing and hashing until we find a matching hash. Once we find a matching hash, we know that the chain for the matching hash will contain the plaintext.

Let us see this in play using the data in the example given before. We are looking to crack "e4ff786adec3a903dc3073720230117d" and get its plaintext. Our database contains just one lonely entry:

a6tyv2 : 6e929c2625cc49bc50d2d1ae845f9cf7
Check if the input hash matches a hash in the database.
Result: Hash not found.

Reduce the input hash (e4ff78), and hash again.
Result: c1b2d3114d814f41638bdafcaeb8e057

Check if the hash matches a hash in the database.
Result: Hash not found.

Reduce the input hash (c1b2d3), and hash again.
Result: 6e929c2625cc49bc50d2d1ae845f9cf7

Check if the hash matches a hash in the database.
Result: Hash found.

Now that we know that the hash belongs to this chain, we start with the initial plaintext (a6tyv2), reduce, and hash our way through the chain until we find the plaintext that gives us our hash.